

AI Exoplanet Habitability Explorer

Model & System Documentation

Final Year Project · University of Agriculture Faisalabad · 2024–2025

Hadeed Ahmad (2022-AG-7746) | Tahzeeb Arif (2022-AG-8065)

Supervisor: Mam Nabeela Ashraf

1. Project Overview

The AI Exoplanet Habitability Explorer is a full-stack web application that applies machine learning to NASA exoplanet transit survey data in order to classify each planet by its potential habitability. The platform targets three distinct audiences: academic researchers who need rapid bulk screening of candidates, students learning about astrobiology, and space enthusiasts who want interactive visualisations of distant worlds.

The system ingests 9,614 confirmed exoplanets observed by three NASA missions — Kepler, K2, and TESS — and classifies each into one of three categories: `POTENTIALLY_HABITABLE`, `HABITABILITY_ZONE`, or `NON_HABITABLE`. Users can also submit custom planet parameters through a prediction panel and receive a real-time habitability score with feature-level explanations.

2. System Architecture

2.1 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	19 / 8	Single-page application
Routing	React Router	v7	Client-side navigation
3D Rendering	React Three Fiber + Drei	9 / 10	WebGL orbital visualisation
Animation	Framer Motion	12	UI transitions
HTTP Client	Axios	1.x	API calls with JWT injection
Backend	Django + DRF	5 / 3	REST API + business logic
Database	Neon DB (PostgreSQL)	16	Cloud-hosted relational DB
ML Runtime	scikit-learn + XGBoost	1.5 / 2	Model inference
Explainability	SHAP + LIME	0.47 / 0.6	Feature attribution
Chatbot LLM	Groq Cloud (Llama 3.3)	70B	ARIA conversational AI

2.2 Request Flow

A browser request follows this path through the system:

- The user interacts with the React frontend served on port 3000 (development) or a CDN edge (production).
- API calls are made to `/api/*` paths. Vite's dev proxy forwards these to the Django backend on port 8000.
- Django URL router dispatches the request to the appropriate Django app (planets, predictions, users, chatbot).
- For prediction requests, the relevant mission-specific ML model (loaded as a singleton at startup) performs inference.
- Results are returned as JSON. The frontend renders them with Framer Motion animations and, where applicable, a 3D React Three Fiber scene.

- Authenticated endpoints require a JWT Bearer token. The Axios interceptor injects the token from localStorage automatically.

2.3 Database Schema (Neon PostgreSQL)

Key tables in the production Neon DB instance:

Table	Key Columns	Notes
planets_planet	id, planet_name, mission_id, pl_rade, pl_masse, pl_eqt, pl_insol, pl_orbper, pl_orbsmax, pl_orbeccen, st_teff, st_rad, st_mass, habitability_class, esi_overall	Core exoplanet catalogue. Populated once from NASA CSV expo
planets_mission	id, name (K2 Kepler TESS)	Foreign-key lookup for mission name.
auth_user	id, username, email, password (bcrypt hash), date_joined, last_login	Django built-in user table. Passwords are never stored in plain tex
users_userprofile	id, user_id (FK), profile_image (TEXT)	Base64 data-URL avatar stored as text.
users_savedprediction	id, user_id (FK), name, inputs (JSON), outputs (JSON), created_at	Each row is one named prediction result saved by a logged-in use

3. Machine Learning Models

3.1 Training Dataset

Each of the three NASA missions received its own dedicated model. This decision was driven by the fact that each mission uses different photometric instruments, cadences, and target-star populations. Training a single cross-mission model would introduce systematic biases from instrument noise that would be incorrectly interpreted as physical planet properties.

Mission	Planets	Potentially Habitable	Best Algorithm	Accuracy	Weighted F1
K2	1,937	8	XGBoost	99.2 %	0.991
Kepler	2,742	11	XGBoost	100 %	1.000
TESS	4,935	9	Random Forest	100 %	1.000
Total	9,614	28	—	—	—

3.2 Input Features

The following eight physical parameters are used as model inputs. All values come from the NASA Exoplanet Archive and were present in sufficient numbers to be useful across all three mission datasets after cleaning.

Feature	Unit	Description
pl_rade	R $\oplus$	Planet radius in Earth radii. Rocky, potentially habitable planets are typically ≤ 2.0 R $\oplus$.
pl_masse	M $\oplus$	Planet mass in Earth masses. Distinguishes rocky from gas-dominated worlds.
pl_eqt	Kelvin	Equilibrium temperature assuming no atmosphere and fixed albedo. Earth ≈ 255 K.
pl_insol	S $\oplus$	Stellar flux received relative to Earth. Habitable zone range is roughly $0.25 - 4.0$ S $\oplus$.
pl_orbper	Days	Orbital period. Determines year length and, combined with semi-major axis, confirms Kepler's third law.
pl_orbsmax	AU	Semi-major axis. Primary determinant of whether a planet falls within the habitable zone.
pl_orbeccen	—	Orbital eccentricity (0 = circular). High values cause extreme seasonal temperature swings.
st_teff	Kelvin	Stellar effective temperature. Determines the habitable zone boundaries and stellar type classification.
st_rad	R $\blacksquare$	Stellar radius in solar radii. Affects luminosity and therefore insolation calculations.
st_mass	M $\blacksquare$	Stellar mass in solar masses. Determines stellar lifetime and orbital dynamics.

3.3 Habitability Classification Labels

- **POTENTIALLY_HABITABLE** — planet satisfies all of: pl_rade ≤ 2.0 R $\oplus$, pl_eqt between 180 K and 320 K, pl_insol between 0.25 and 4.0 S $\oplus$, and falls within the computed Kopparapu habitable zone. Only 28 of 9,614 planets meet all criteria simultaneously.
- **HABITABILITY_ZONE** — planet orbits within the habitable zone distance but fails at least one physical criterion (e.g. radius too large, temperature marginal). These are secondary candidates worth further observation.

- **NON_HABITABLE** — planet does not meet the orbital or physical criteria. This is the overwhelming majority of detected exoplanets, including hot Jupiters, lava worlds, and frozen ice giants.

3.4 Training Procedure

For each mission the dataset was split 80 % training / 20 % test with stratification to preserve the class ratios. Because **POTENTIALLY_HABITABLE** represents fewer than 0.3 % of samples, SMOTE (Synthetic Minority Over-sampling Technique) was applied to the training fold only to prevent the model from simply predicting **NON_HABITABLE** for every input.

Hyperparameter optimisation used 5-fold stratified cross-validation with GridSearchCV. The final selected parameters for XGBoost were: `n_estimators=200`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`. The Random Forest used `n_estimators=500`, `max_depth=20`, `min_samples_split=2`.

All three trained model objects are serialised as pickle files stored in the backend `models/` directory and loaded once at Django startup via a singleton `HabitabilityScorer` class. The scorer accepts mission name + feature dictionary and routes the call to the correct model.

3.5 Composite Habitability Score

The raw ML class probability is combined with three independent sub-scores to produce a single 0–100 composite score displayed in the prediction panel:

Component	Weight	Source
ML classification prob	40 %	Mission-specific XGBoost / Random Forest output
Earth Similarity Index	30 %	Radius + temperature + flux similarity to Earth
HZ proximity score	20 %	Normalised distance to the Kopparapu HZ centre
Stellar factor	10 %	Penalty for stars that are too hot, too cool, or too short-lived

4. REST API Reference

4.1 Planet Endpoints

Method	Path	Auth	Description
GET	/api/planets/	No	Paginated planet list. Query params: page, page_size, mission, habitability, q (search)
GET	/api/planets/{id}/	No	Full detail for a single planet including all physical parameters.
GET	/api/planets/stats/	No	Aggregate counts by mission and habitability class.
GET	/api/missions/	No	List of available missions (K2, Kepler, TESS).

4.2 Prediction Endpoints

Method	Path	Auth	Description
POST	/api/predict/	No	Single-planet prediction. Body: { mission, pl_rade, pl_masse, pl_eqt, pl_insol, pl_age, pl_hab }.
POST	/api/predict/batch/	No	Batch predictions. Body: { planets: [...same as single...] }

4.3 Authentication Endpoints

Method	Path	Auth	Description
POST	/api/auth/register/	No	Create account. Body: { username, email, password }
POST	/api/auth/login/	No	Login with username or email. Returns { access, refresh } JWT pair.
GET	/api/auth/me/	JWT	Current user profile including username, email, and profile image.
GET	/api/auth/saved/	JWT	List all saved predictions for the authenticated user.
POST	/api/auth/saved/	JWT	Save a prediction. Body: { name, inputs, outputs }
DELETE	/api/auth/saved/{id}/	JWT	Delete a specific saved prediction by ID.

4.4 Chatbot Endpoint (ARIA)

Method	Path	Auth	Description
GET	/api/chatbot/	No	Status check — returns provider, active model, and whether GROQ_API_KEY is configured.
POST	/api/chatbot/	No	Send message. Body: { message: string, history: [{ role, content }] }. Returns { reply, message_id }.

5. Frontend Pages & Components

5.1 Page Inventory

Route	Component	Description
/	Home.jsx	Landing page with hero section, feature cards, stats, and call-to-action links.
/explore	ExplorePlanets.jsx	Main explorer: filter sidebar, 3D orbital viewer trigger, paginated planet grid, and pre-
/planet/:id	PlanetDetail.jsx	Full detail view for a single planet with all parameters and ML prediction with SHAP e
/solar-system	SolarSystem.jsx	Interactive 3D Solar System viewer with all 8 planets, moons, asteroids, and Artemis
/predict	Predict.jsx	Standalone habitability prediction form for custom planet parameters.
/about	About.jsx	This page — project overview, methodology, team, and documentation downloads.
/login	Login.jsx	JWT login form. Accepts username or email.
/register	Register.jsx	Account creation form with profile image upload.
/profile	Profile.jsx	User profile and saved predictions history.

5.2 3D Visualisation Architecture

The ExoplanetViewer3D component uses React Three Fiber (r3f) and Drei to render a real-time WebGL orbital scene inside a modal overlay. The scene is driven entirely by live API data filtered to a maximum of 28 planets for 60 fps performance.

- CentralStar — temperature-coloured sphere with pulsing corona. Colour ranges from red (M-dwarf < 3000 K) through orange (K-type), yellow (G-type), white (F-type) to blue (A/B/O-type).
- OrbitPath — a Drei Line component drawing 256-point ellipses using the planet's semi-major axis and eccentricity. Each orbit has a subtle inclination derived from the planet index.
- OrbitalPlanet — a Three.js mesh that orbits at the correct angular velocity. Texture is selected by equilibrium temperature: frozen / ice / temperate / lava / gas. Gas giants ($p_l\text{_rade} > 3.5 R_{\oplus}$) have a PlanetRing torus geometry. Hovering increases emissive intensity from 0.5 to 1.1.
- Habitable Zone Ring — a green ring around the star rendered using the Kopparapu 2013 inner/outer HZ boundaries scaled by stellar temperature.
- Camera modes — Top View lerps the camera to [0, 52, 0.5]; Surface View zooms to [0, 4, 10] when a planet is selected; Reset returns to the default perspective. Auto-rotate pauses on user interaction via an interactingRef flag.

5.3 Authentication Flow

- User submits login form → POST /api/auth/login/ → receives { access, refresh } JWT pair.
- Tokens are written to localStorage under keys 'access_token' and 'refresh_token'.
- An Axios request interceptor reads 'access_token' from localStorage and injects it as Authorization: Bearer on every outgoing request automatically.
- An Axios response interceptor catches 401 responses, clears localStorage, and redirects to /login.
- The /api/auth/me/ endpoint is called on app load to hydrate the current user state.

6. Known Limitations

These limitations are scientific and data constraints — they are not bugs or implementation gaps in the system.

- Atmospheric composition is completely absent from the dataset. Transit photometry measures how much starlight a planet blocks, not what its atmosphere contains. Determining whether a planet has oxygen, water vapour, or carbon dioxide requires transmission spectroscopy, currently only feasible with JWST for a handful of targets.
- `pl_eqt` is not actual surface temperature. It assumes zero greenhouse effect and a fixed albedo. Venus has `pl_eqt` ≈ 232 K but an actual surface temperature of 465°C due to its dense CO_2 atmosphere. Our model cannot distinguish this case.
- Only 28 `POTENTIALLY_HABITABLE` training samples exist across all three missions. The positive class is extremely sparse, which limits the model's ability to generalise to novel habitable candidates that sit far from the known 28 in feature space.
- No magnetic field data is available. A planetary magnetic field is widely considered necessary to shield a surface biosphere from stellar wind particle radiation, particularly for close-in planets around active M-dwarf stars.
- No tidal locking assessment. Planets with short orbital periods around cool stars are likely tidally locked, which concentrates irradiation on one hemisphere. This is not captured by any current feature.
- No geological activity proxy. Active volcanism recycles atmospheric carbon and drives the carbonate-silicate cycle that stabilises long-term climate, but no proxy for this is available in transit data.

7. Deployment Notes

7.1 Environment Variables (`backend/.env`)

Variable	Required	Description
<code>SECRET_KEY</code>	Yes	Django secret key for CSRF and session signing.
<code>DEBUG</code>	Yes	Set to False in production.
<code>DB_NAME</code>	Yes	Neon PostgreSQL database name.
<code>DB_USER</code>	Yes	Neon PostgreSQL username.
<code>DB_PASSWORD</code>	Yes	Neon PostgreSQL password.
<code>DB_HOST</code>	Yes	Neon pooler hostname.
<code>DB_PORT</code>	Yes	PostgreSQL port (default 5432).
<code>DATABASE_URL</code>	Yes	Full connection string (alternative to individual <code>DB_*</code> vars).
<code>GROQ_API_KEY</code>	Yes	Groq Cloud API key for ARIA chatbot. Obtain free at console.groq.com .

7.2 Running Locally

- Backend: `cd backend && pip install -r requirements.txt && python manage.py migrate && python manage.py runserver`
- Frontend: `cd frontend && npm install && npm run dev`
- Both servers must run simultaneously. The Vite proxy forwards `/api/*` calls from port 3000 to port 8000.
- To access the Django admin panel: `python manage.py createsuperuser`, then visit `http://localhost:8000/admin/`

Document generated automatically · AI Exoplanet Habitability Explorer · 2025